



CI/CD

Infrastructure as Code
(IaC)



Outline

- 1 **laC**
- 2 **Terraform**

1

Infrastructure as Code



Apa itu IaC?



Infrastructure as Code (IaC) adalah pendekatan otomatisasi infrastruktur TI di mana pengelolaan dan penyediaan sumber daya seperti server, jaringan, dan storage dilakukan melalui file konfigurasi yang dapat dijalankan otomatis, bukan melalui proses manual. Dengan menggunakan script atau template, IaC memungkinkan infrastruktur didefinisikan, dilacak versinya, diuji, dan di-deploy secara konsisten layaknya kode aplikasi, sehingga meningkatkan efisiensi, mengurangi human error, dan mempermudah replikasi lingkungan dalam skala besar.



Manfaat IaC

 **Repeatable:** Infrastruktur dapat direplikasi secara konsisten.

 **Version Control:** Konfigurasi disimpan di Git, bisa audit & rollback.

 **Deployment Cepat:** Otomatisasi provisioning server dan resource cloud.

 **Testable:** Validasi konfigurasi sebelum diterapkan.

 **Efisiensi Biaya:** Mengurangi human error dan idle resource.

Prinsip Dasar IaC

Idempotency: Eksekusi berulang menghasilkan hasil yang sama.

Version Control: Konfigurasi dicatat dan diaudit.

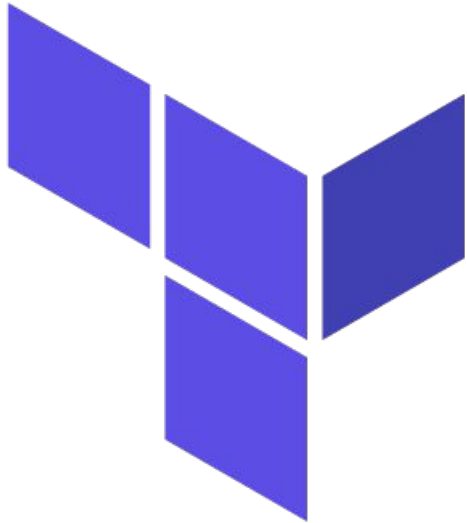
Automation First: Tidak ada intervensi manual.

Self-Documentation: Kode = dokumentasi konfigurasi.

Tantangan IaC

- Kompleksitas pada skala besar.
- Memerlukan skill scripting & toolset baru.
- Salah kode = dampak luas (kerusakan massal).

IaC Solution



HashiCorp

Terraform



Pulumi

Masalah

Sebuah tim developer butuh membuat server uji coba (VM + Database) untuk setiap fitur baru. Sebelumnya dilakukan **secara manual** melalui dashboard cloud, memakan waktu 1–2 jam dan sering terjadi konfigurasi yang tidak konsisten.

Solusi IaC

- Buat template konfigurasi server & database dalam file YAML/JSON.
- Gunakan Git untuk version control.
- Jalankan provisioning otomatis setiap ada branch fitur baru.

Hasil

- Setup server hanya butuh 5 menit secara otomatis.
- Semua environment identik (dev, staging, testing).
- Perubahan konfigurasi bisa di-review dan di-rollback via Git.

2

Terraform



Apa itu Terraform

Terraform adalah tool open-source yang digunakan untuk mendefinisikan, mengelola, dan menyebarkan infrastruktur secara otomatis dan konsisten melalui pendekatan Infrastructure as Code (IaC).



Apa itu Terraform

- ✓ Dikembangkan oleh **HashiCorp**.
- ✓ Mendukung berbagai provider: AWS, Azure, GCP, Kubernetes, dan lainnya.
- ✓ File konfigurasi berbasis **HCL (HashiCorp Configuration Language)**.

Kenapa Terraform ?

- Mendukung **multi-cloud** dan hybrid cloud.
- **Deklaratif**: cukup sebutkan state yang diinginkan.
- **Modular**: konfigurasi dapat dipecah dan digunakan ulang.
- Tersedia fitur **plan, validate, dan apply**.
- Kompatibel dengan GitOps & CI/CD pipeline.

Arsitektur Terraform ?

- File .tf (Terraform Configuration)
- Terraform CLI
- State File (terraform.tfstate)
- Provider Plugin (AWS, Azure, dll.)
- Backend (lokal, remote seperti S3)

Terraform Lifecycle



Write: Definisikan resource dalam .tf

Init: Inisialisasi project dan plugin provider

Plan: Simulasi perubahan infrastruktur

Apply: Terapkan perubahan

Destroy: Hapus semua resource

Best Practice

- Jangan simpan secrets dalam file `.tf` (gunakan Vault)
- Gunakan remote state dengan backend aman (S3 + DynamoDB, GCS)
- Selalu lakukan `plan` sebelum `apply`
- Gunakan modul untuk reusable code

3

Hands On



Installation

Kamu bisa melihat pada dokumentasi resmi Terraform disini untuk melakukan instalasi <https://developer.hashicorp.com/terraform/install/> , silahkan disesuaikan berdasarkan operating system yang digunakan



```
brew tap hashicorp/tap  
brew install hashicorp/tap/terrafor
```

Studi Kasus

Seorang DevOps engineer ingin mendeploy container berbasis image **nginx:latest** ke **Google Cloud Run** dengan domain publik dan tanpa autentikasi, serta dikelola sepenuhnya menggunakan **Terraform**.



Studi Kasus

Struktur Folder project

```
cloud-run-nginx/  
├── main.tf  
├── variables.tf  
├── outputs.tf  
└── terraform.tfvars
```

Persyaratan

Sudah mengaktifkan **Cloud Run API** dan **IAM API**

Sudah memiliki **Google Cloud Project ID**

Sudah mengatur **gcloud auth
application-default login**

Sudah install **Terraform**

Step by Step

file main.tf kamu bisa melihat di pastebin

<https://pastebin.com/X8gh6gud>

Step by Step

file variables.tf

```
variable "project_id" {  
    description = "Google Cloud Project ID"  
    type        = string  
}  
  
variable "region" {  
    description = "Region to deploy Cloud Run"  
    type        = string  
    default     = "us-central1"  
}
```

Step by Step

file terraforms.tfvars



```
project_id = "your-gcp-project-id"  
region     = "us-central1"
```

Step by Step

file outputs.tf



```
output "cloud_run_url" {  
  value =  
  google_cloud_run_service.nginx.status[0].url  
}
```

Step by Step

Langkah - langkah menjalankan Terraform

1. **Init** terraform

\$ terraform init

2. **Review** plan

\$ terraform review

3. **Deploy**

\$ terraform apply

4. **Output**

URL endpoint NGINX tersedia di output `cloud_run_url`

Akses publik tanpa autentikasi

Hasil akhir

✓ Hasil yang Dicapai

- NGINX image berjalan di container Cloud Run
- Dapat diskalakan otomatis (autoscaling)
- Dikelola sepenuhnya oleh Terraform (idempotent dan reproducible)
- Akses publik via HTTPS



</Rakamin

Terima kasih